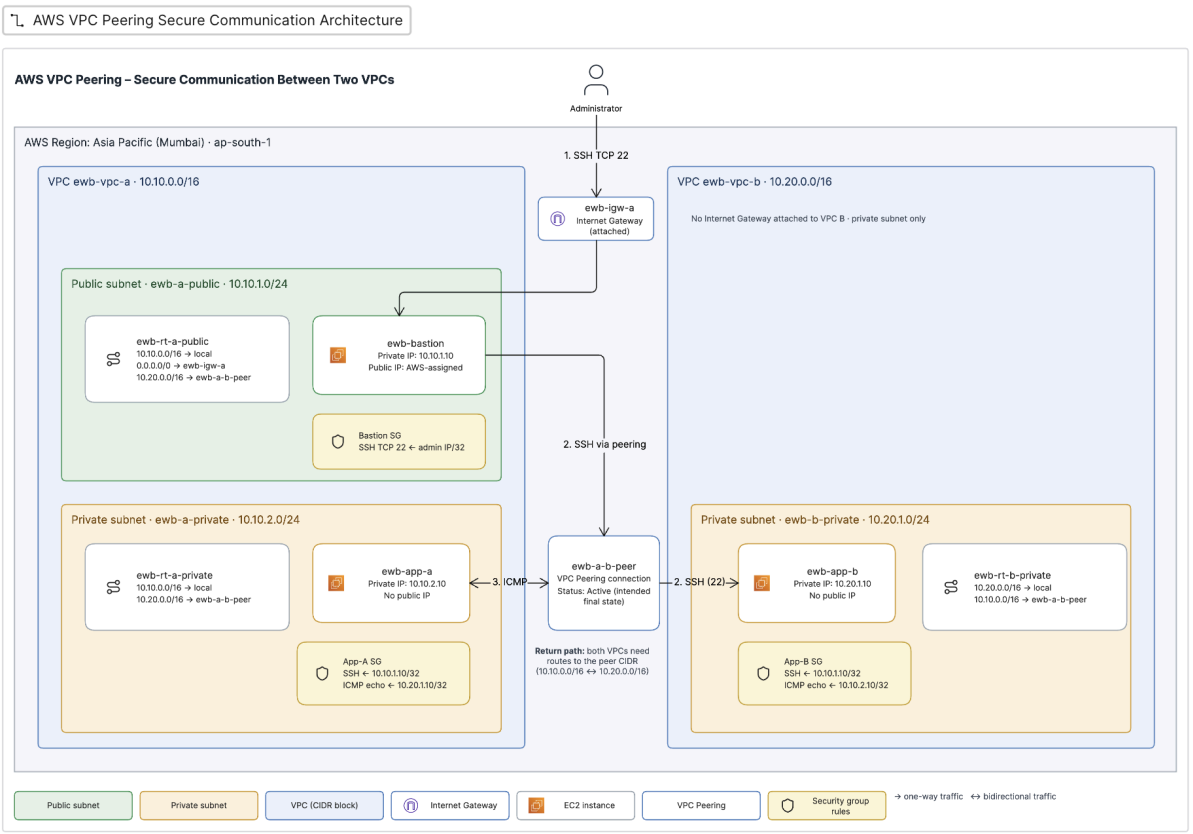
AWS VPC PEERING PROJECT REPORT

Name:- Sanyukta Nilesh Chewale
Batch:- AWS Architect Associate - OCT 2026
Project Title:- AWS VPC PEERING
Region:- Asia Pacific (Mumbai) — ap-south-1
Completion Date :- 24 October 2026

INTRODUCTION

Amazon VPC allows users to create logically isolated virtual networks in AWS. Within a VPC, users can configure IP address ranges, subnets, route tables, gateways, and security controls.
In this project, two VPCs are created to represent separate application environments. The first VPC contains a bastion host and a private application server. The second VPC contains another private application server.
VPC Peering connects the two VPCs so that resources can communicate using private IPv4 addresses, provided that appropriate routes and security rules are configured.
The application instances remain private and are not assigned public IPv4 addresses. Administrative SSH connections are made through the bastion host in VPC A.

NETWORK ARCHITECTURE



The architecture consists of two VPCs connected through a VPC peering connection.
VPC A contains one public subnet and one private subnet. The public subnet contains the bastion host, while the private subnet contains App-A.VPC B contains one private subnet with App-B. VPC B does not require an internet gateway for the private communication demonstrated in this project.The bastion host is used as an administrative entry point. The two application servers communicate directly over the peering connection using their private IP addresses.

Resource Inventory Table

Resource	Name	IPv4 CIDR / private IP	Actual AWS resource ID
VPC A	ewb-vpc-a	10.10.0.0/16	vpc-02dadd238e580fbf6
VPC B	ewb-vpc-b	10.20.0.0/16	vpc-056babb9b155d15ed
Public subnet A	ewb-a-public	10.10.1.0/24	subnet-043068acc72aef178
Private subnet A	ewb-a-private	10.10.2.0/24	subnet-0dc82d5aa60465934
Private subnet B	ewb-b-private	10.20.1.0/24	subnet-016e6248e9f5901d0
Internet gateway	ewb-igw-a	N/A	igw-04d7c9680271c1244
Public route table	ewb-rt-a-public	N/A	rtb-0ca973263ed9e2fbc
Private route table A	ewb-rt-a-private	N/A	rtb-02ba7d4460bd3b547
Private route table B	ewb-rt-b-private	N/A	rtb-0dfd19ad95bbcba53
Bastion EC2	ewb-bastion	10.10.1.10	i-09cdf1b3a642cee52
App-A EC2	ewb-app-a	10.10.2.10	i-0f4285a746a271d32
App-B EC2	ewb-app-b	10.20.1.10	i-0e3b521b75b5d089b
VPC peering	ewb-a-b-peer	Active(pcx)	pcx-07b4cb215f9ee1782

NETWORK IMPLEMENTATION

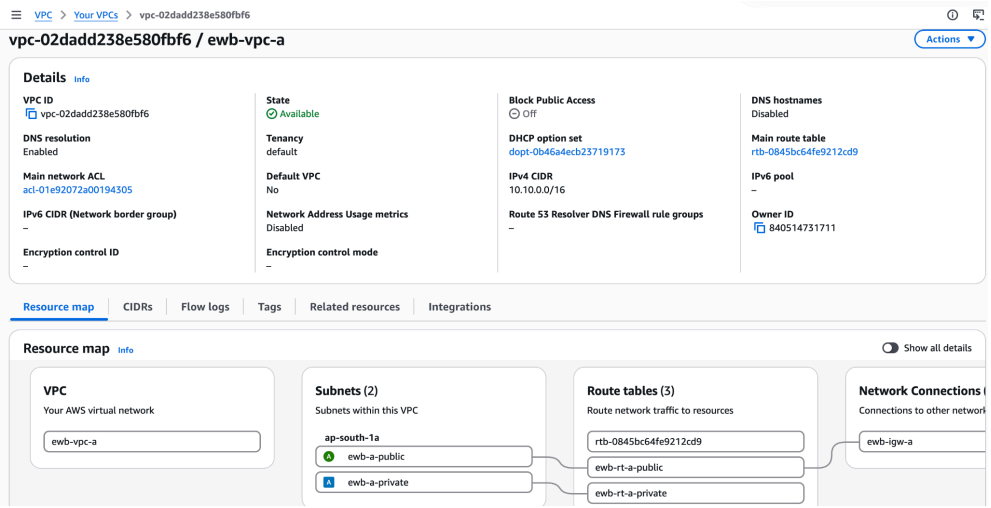
1. Creating VPC A

Configuration:

Name: ewb-vpc-a

IPv4 CIDR: 10.10.0.0/16

Region: ap-south-1



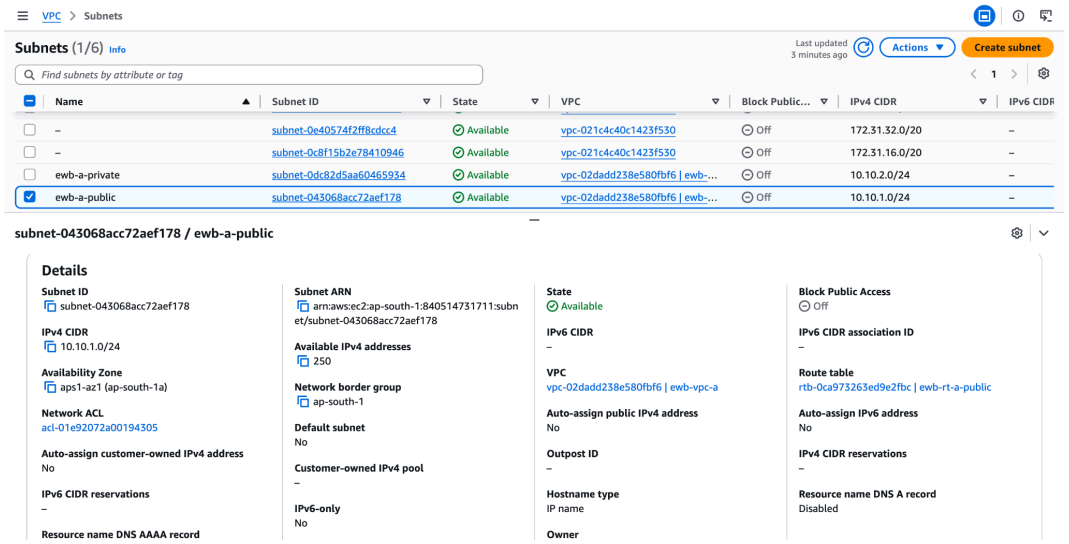
Creating the Subnets in VPC A

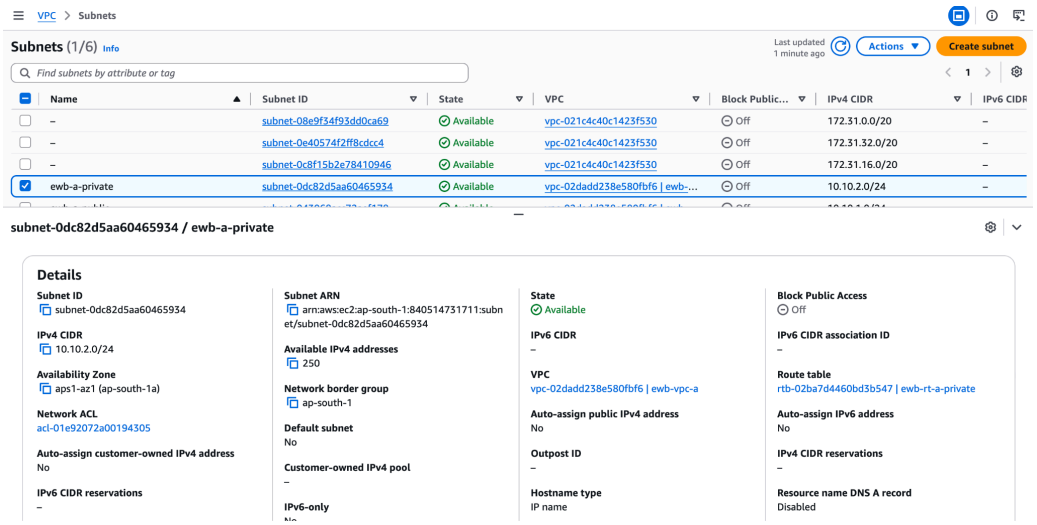
Two subnets were configured in VPC A.

The public subnet, ewb-a-public, uses CIDR 10.10.1.0/24 and hosts the bastion instance. The private subnet, ewb-a-private, uses CIDR 10.10.2.0/24 and hosts App-A.

The public subnet is associated with the public route table. The private subnet is associated with the private route table.

Automatic public IPv4 assignment is disabled for the private subnet.





2. Creating VPC B

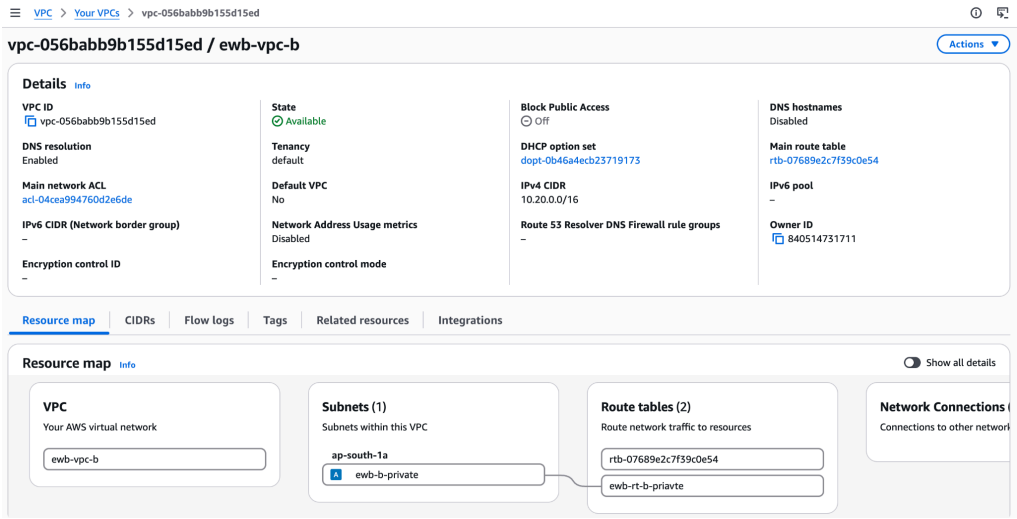
Configuration:

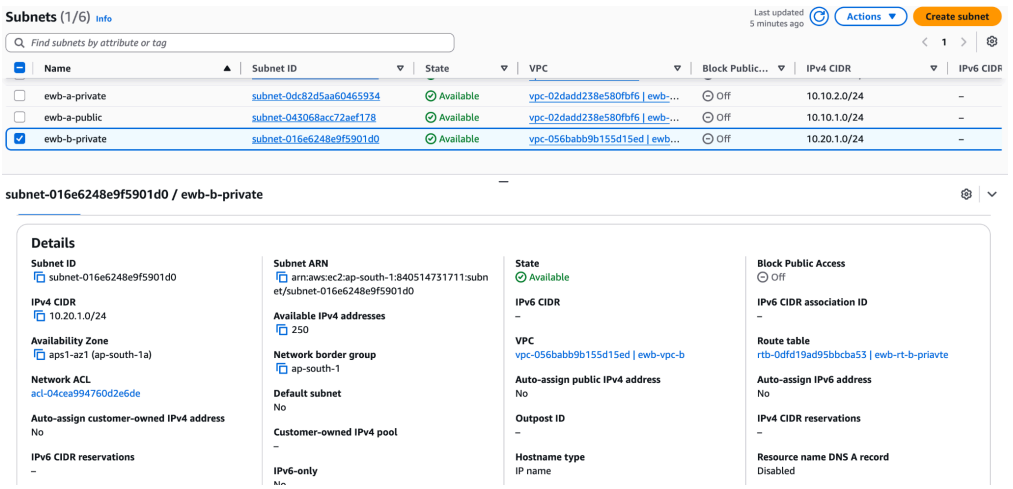
Name: ewb-vpc-b

IPv4 CIDR: 10.20.0.0/16

Private subnet: ewb-b-private

Subnet CIDR: 10.20.1.0/24





ROUTE TABLE CONFIGURATION

Route tables determine how traffic is forwarded from a subnet. This project uses three custom route tables.

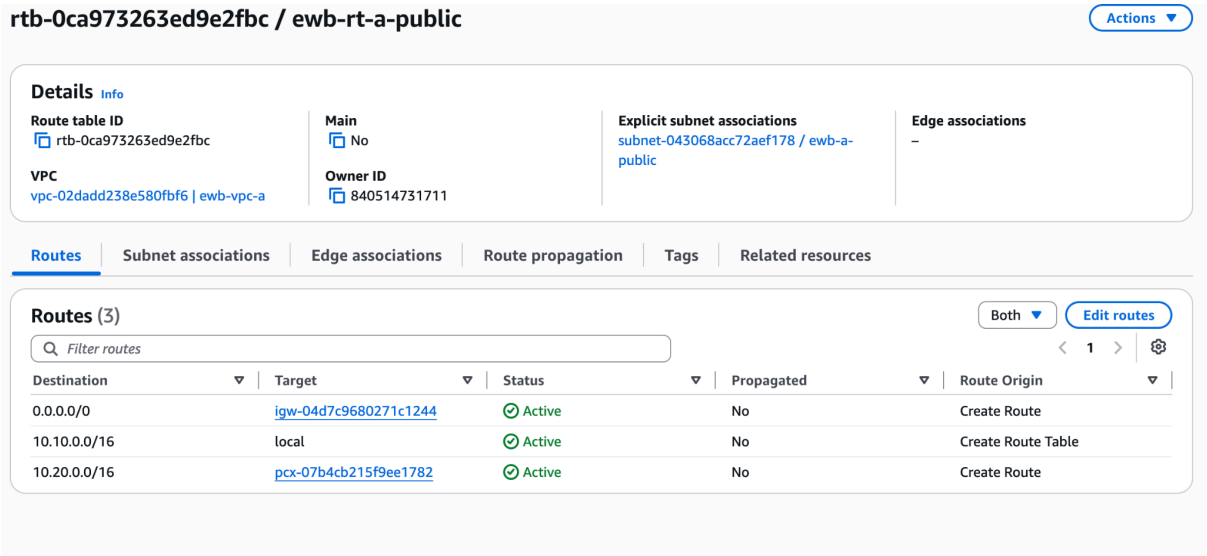
Each route table retains its AWS-created local route. The peering routes provide paths between the two VPCs, while the internet gateway route provides internet access for the bastion subnet.

1. Public Route Table — ewb-rt-a-public

Subnet association: ewb-a-public

Destination	Target	Purpose
10.10.0.0/16	local	Communication within VPC A
0.0.0.0/0	ewb-igw-a	Internet access for the public subnet
10.20.0.0/16	ewb-a-b-peer (pcx-...)	Reach VPC B from the bastion subnet

The route to 10.20.0.0/16 is required because the bastion opens a separate SSH connection to App-B when SSH ProxyJump is used. The bastion subnet must have a route to VPC B through the peering connection.

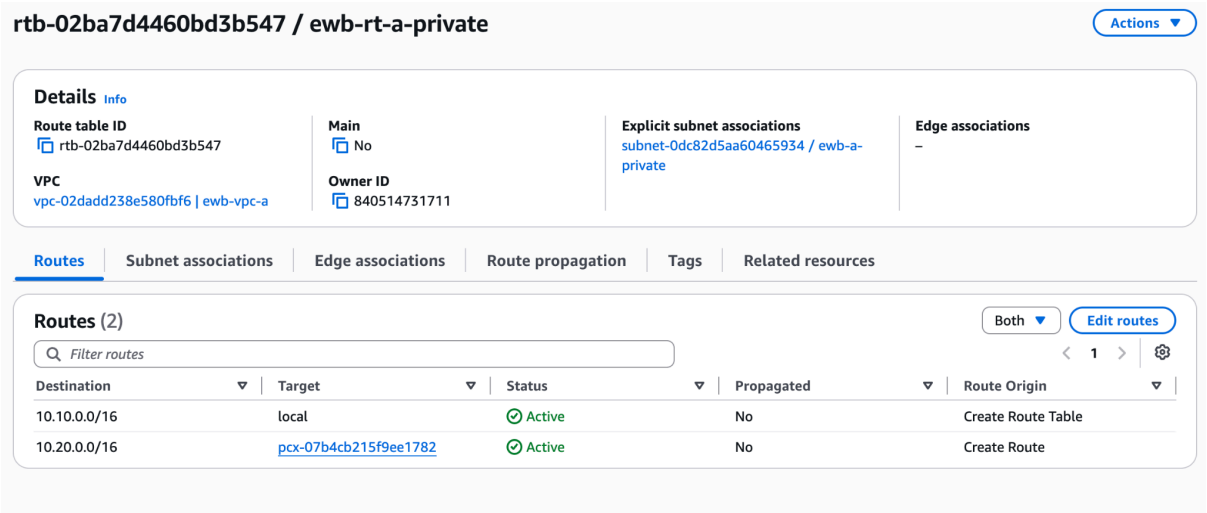


Private Route Table A — ewb-rt-a-private

Subnet association: ewb-a-private

Destination	Target	Purpose
10.10.0.0/16	local	Communication within VPC A
10.20.0.0/16	ewb-a-b-peer (pcx-...)	Send App-A traffic to VPC B

This route allows App-A to send traffic to App-B's private IP address through the peering connection.

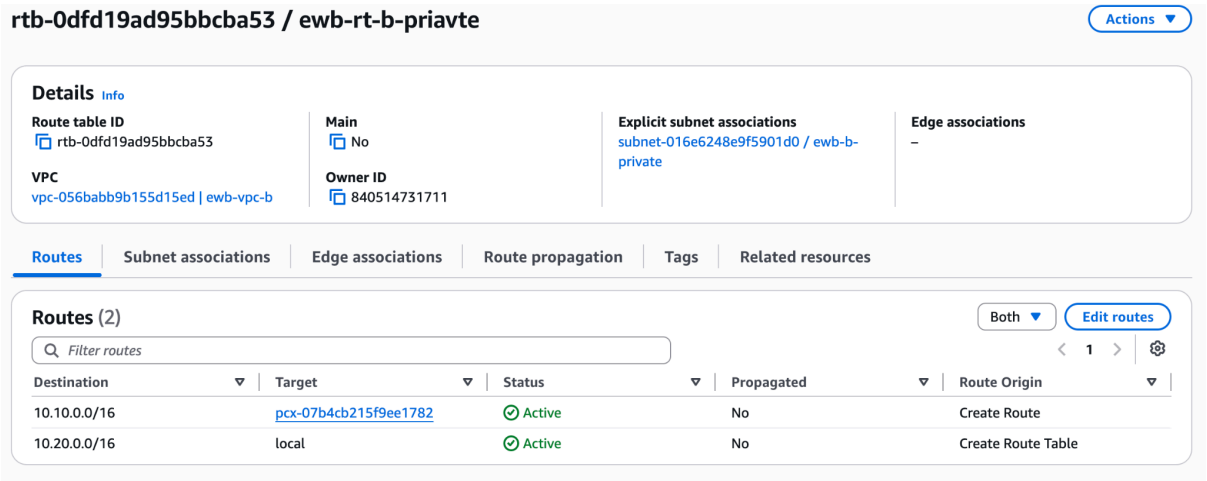


Private Route Table B — ewb-rt-b-private

Subnet association: ewb-b-private

Destination	Target	Purpose
10.20.0.0/16	local	Communication within VPC B
10.10.0.0/16	ewb-a-b-peer (pcx-...)	Return traffic to VPC A

The return route is necessary because communication requires both a forward path and a return path. A route in only one VPC is not sufficient for successful two-way communication.



SECURITY GROUP CONFIGURATION

Security groups provide instance-level traffic filtering. The inbound rules are restricted to the sources required for this lab.

The project uses three security groups: **ewb-sg-bastion**, **ewb-sg-app-a**, and **ewb-sg-app-b**.

1. Bastion Security Group — ewb-sg-bastion

Type	Protocol / port	Source	Purpose
SSH	TCP 22	Current public IPv4 /32	Allow SSH from the current public IPv4 approved client

2. App-A Security Group — ewb-sg-app-a

Type	Protocol / port	Source	Purpose
SSH	TCP 22	10.10.1.10/32	Allow SSH from the bastion

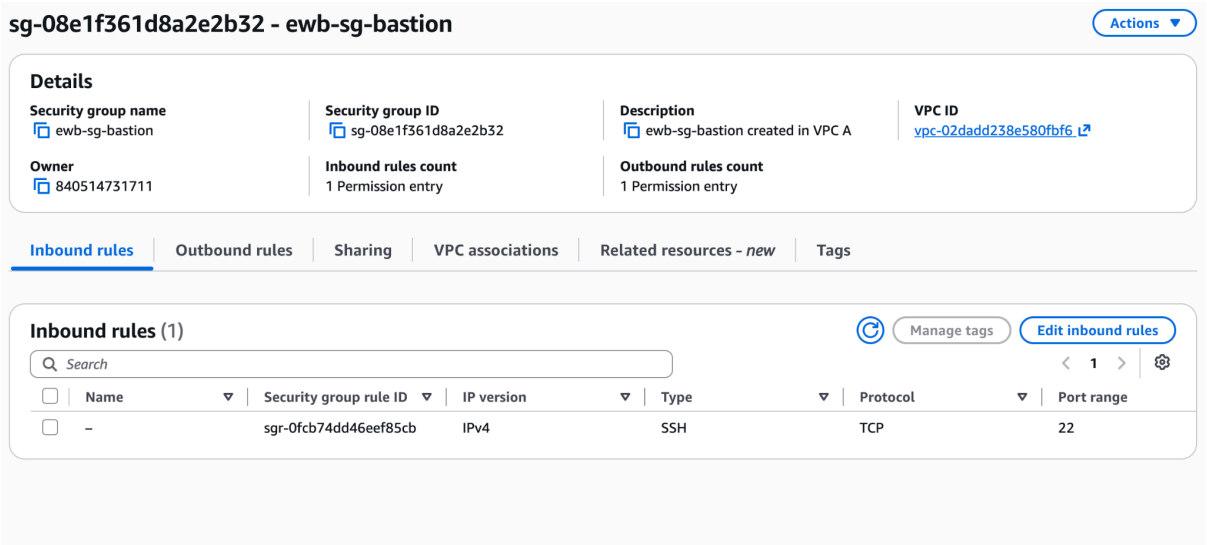
Custom ICMP - IPv4	Echo request, type 8, code 0	10.20.1.10/32	Allow ping requests from App-B

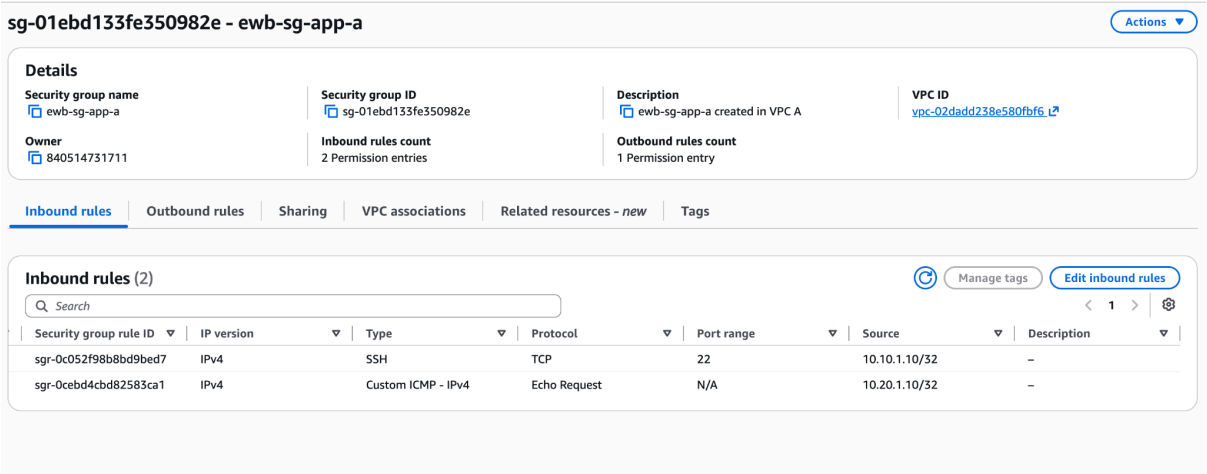
3. App-B Security Group — ewb-sg-app-b

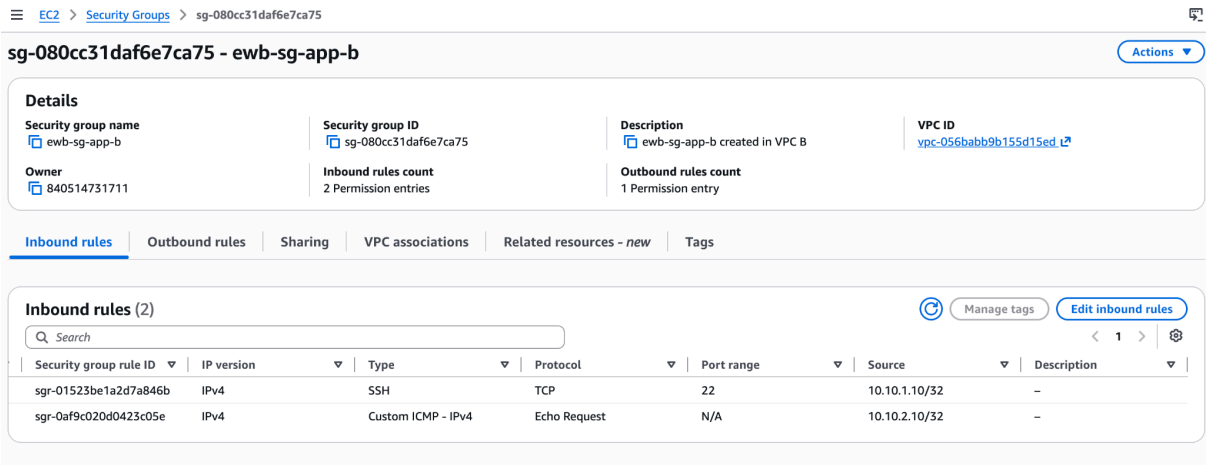
Type	Protocol / port	Source	Purpose
SSH	TCP 22	10.10.1.10/32	Allow SSH from the bastion
Custom ICMP - IPv4	Echo request, type 8, code 0	10.10.2.10/32	Allow ping requests from App-A

Security groups are stateful, so responses to permitted connections are allowed automatically. The inbound ICMP rules are still required for the destination instances to receive ping requests.

The SSH source on App-B is the bastion private IP because the bastion initiates the SSH connection to App-B through ProxyJump.







VPC PEERING CONFIGURATION

Creating the Peering Connection

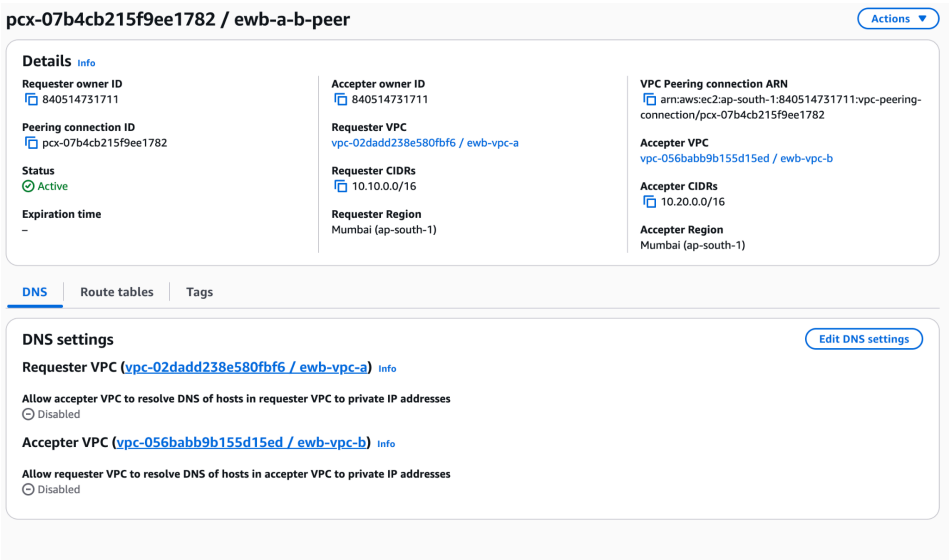
A VPC peering connection named ewb-a-b-peer was planned between VPC A and VPC B.

The requester is ewb-vpc-a, and the accepter is ewb-vpc-b. Both VPCs are in the same AWS account and Region for this project.

Accepting the Request

After creating the peering request, the accepter request must be accepted from the VPC console. The connection must show the status Active before connectivity testing.

An active peering connection does not automatically create routes or permit traffic. The required route-table entries and security-group rules must also be present.

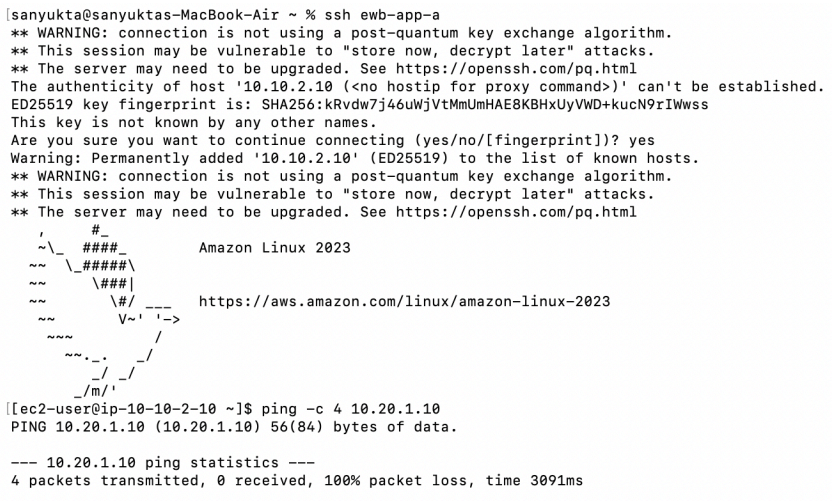


Verification and Test Results

Test 1 : Baseline Failure Before Peering Routes

From App-A, run: ping -c 4 10.20.1.10

Expected outcome: before the required peering routes are configured, App-A should not receive ping replies from App-B. Record the actual output and packet loss.

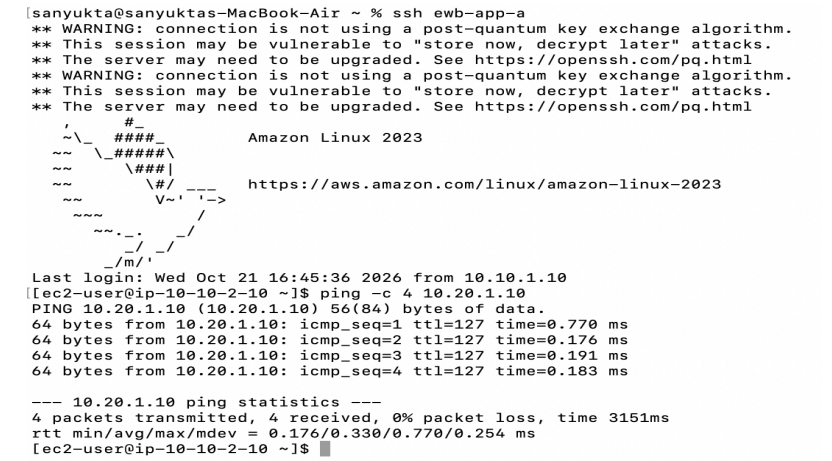


Test 2 : Forward Ping: App-A to App-B

After peering is Active and routes/security rules are configured, run from App-A: ping -c 4 10.20.1.10

Expected outcome: four replies and 0% packet loss, if the route and security rules are correct.

Forward path: App-A sends traffic to 10.20.1.10. Its subnet route table matches 10.20.0.0/16 and forwards the packet to the peering connection. App-B's security group must allow inbound ICMP Echo Request from 10.10.2.10/32.

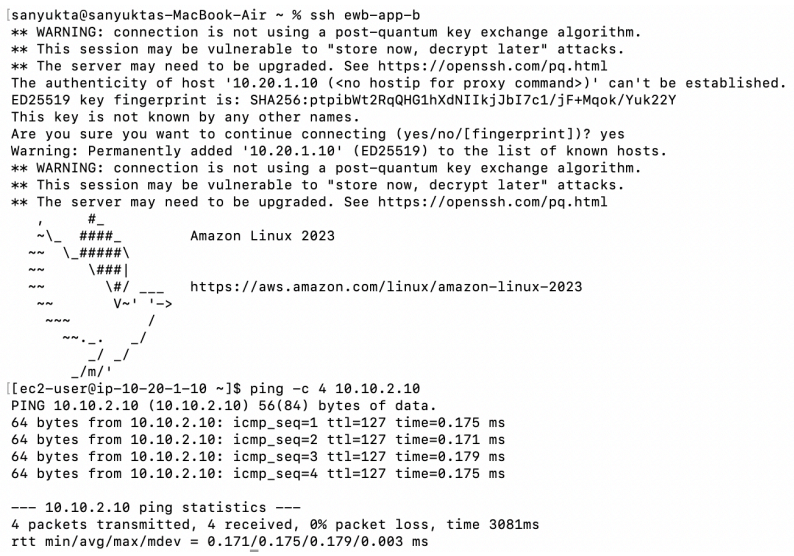


Test 3 : Reverse Ping: App-B to App-A

From App-B, run: ping -c 4 10.10.2.10

Expected outcome: four replies and 0% packet loss, if the reverse route and App-A's inbound ICMP rule are correct.

Path: App-B's subnet route table matches 10.10.0.0/16 and sends traffic through the peering connection. App-A's security group must allow inbound ICMP Echo Request from 10.20.1.10/32. Security groups are stateful, allowing responses for permitted flows.urity group on App-A must permit ICMP echo requests from App-B.



Test 4 : SSH to App-B Through the Bastion

run: ssh ewb-app-b

Expected outcome: a shell connection to App-B through the bastion, if SSH configuration, key, routes, and security rules are correct.

SSH path: the laptop connects to the bastion's public IPv4 address on TCP port 22. The bastion initiates a separate SSH connection to App-B at 10.20.1.10. The bastion subnet needs a route to 10.20.0.0/16 through peering. App-B must allow TCP 22 from 10.10.1.10/32, and VPC B needs a return route to 10.10.0.0/16.

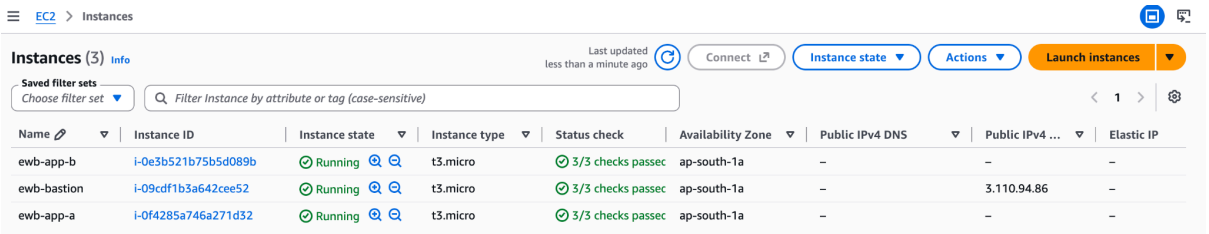
```
sanyukta@sanyuktas-MacBook-Air ~ % ssh ewb-app-b
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
Last login: Wed Oct 21 17:26:07 2026 from 10.10.1.10
[ec2-user@ip-10-20-1-10 ~]$ ping -c 4 10.10.2.10
PING 10.10.2.10 (10.10.2.10) 56(84) bytes of data.
64 bytes from 10.10.2.10: icmp_seq=1 ttl=127 time=0.255 ms
64 bytes from 10.10.2.10: icmp_seq=2 ttl=127 time=0.169 ms
64 bytes from 10.10.2.10: icmp_seq=3 ttl=127 time=0.167 ms
64 bytes from 10.10.2.10: icmp_seq=4 ttl=127 time=0.173 ms

--- 10.10.2.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3092ms
rtt min/avg/max/mdev = 0.167/0.191/0.255/0.037 ms
[ec2-user@ip-10-20-1-10 ~]$ exit
```

Test 5 : Private Address Verification

In the EC2 console, inspect the networking details for App-A and App-B.

Expected outcome: neither App-A nor App-B has a public IPv4 address. Their private IPs should match the addresses used in the tests.

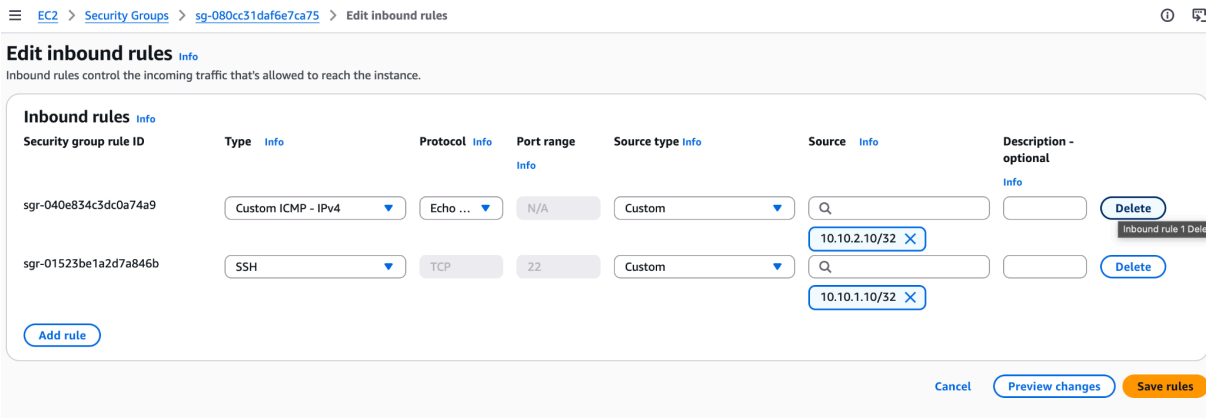


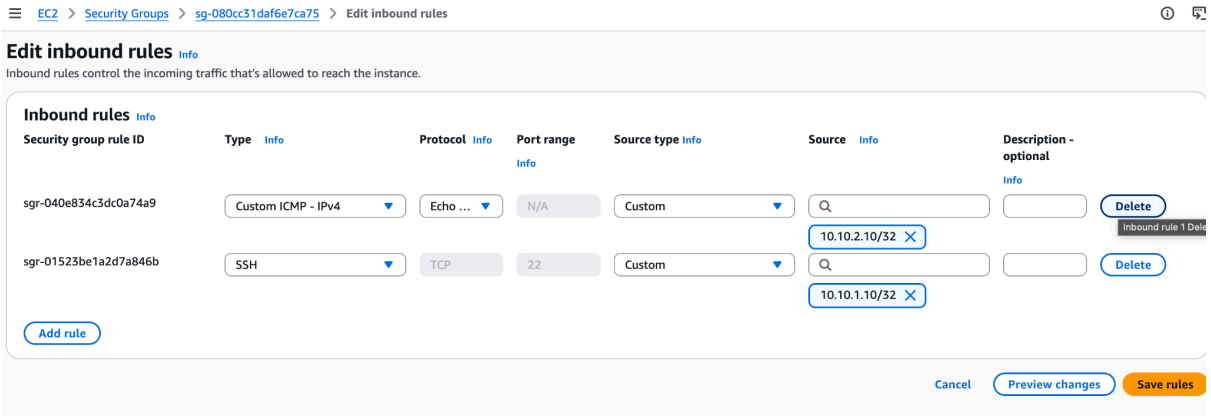
Test 6 : Controlled ICMP Failure

Temporarily remove only the inbound ICMP Echo Request rule on ewb-sg-app-b that permits source 10.10.2.10/32. Keep SSH unchanged. From App-A, run: ping -c 4 10.20.1.10

Expected outcome: App-A should receive no ping replies while App-B's inbound ICMP rule for App-A is absent, assuming other settings remain unchanged.

Explanation: the peering route remains available, but App-B's security group no longer permits inbound ICMP Echo Requests from App-A.






```
sanyukta@sanyuktas-MacBook-Air ~ % ssh ewb-app-a
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
The authenticity of host '10.10.2.10 (<no hostip for proxy command>)' can't be established.
ED25519 key fingerprint is: SHA256:kRvdw7j46uWjVtMmUmHAE8KBHxUyVWD+kucN9rIWwss
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.10.2.10' (ED25519) to the list of known hosts.
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
[ec2-user@ip-10-10-2-10 ~]$ ping -c 4 10.20.1.10
PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.

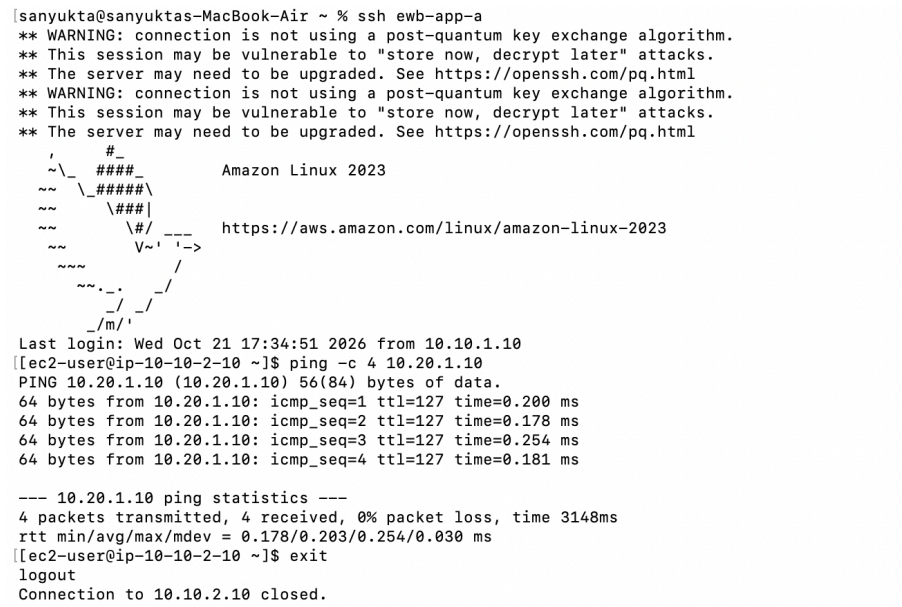
--- 10.20.1.10 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3091ms
```

Test 7 : Recovery After Restoring ICMP

Restore the inbound ICMP Echo Request rule on ewb-sg-app-b with source 10.10.2.10/32. From App-A, run: ping -c 4 10.20.1.10

Expected outcome: ping replies resume, ideally four replies and 0% packet loss.

Explanation: restoring the rule allows App-B's security group to accept ICMP Echo Requests from App-A.



Viva Questions

1. Why is the return route needed?

The return route allows App-B to send replies back to App-A. Without it, the request may reach App-B, but the response cannot return. Both VPCs need routes to each other through the peering connection.

2. Why is the App-B SSH source the bastion private IP?

App-B allows SSH only from the bastion's private IP, 10.10.1.10/32. This restricts access to the authorized bastion host and improves security. It follows the principle of least privilege.

3. Why does Active peering not guarantee a successful ping?

Active status means the peering connection is established. Ping can still fail due to missing routes, security-group rules, or network ACL restrictions. Both routing and security settings must allow ICMP traffic.

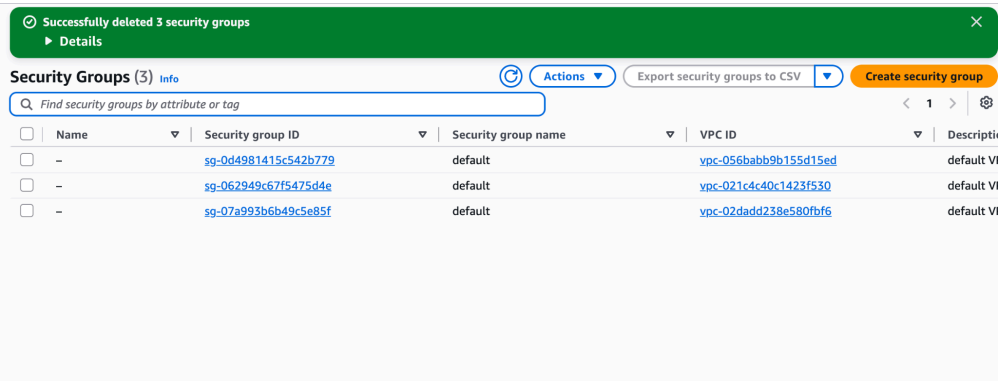
4. Can VPC B use VPC A's internet gateway over peering?

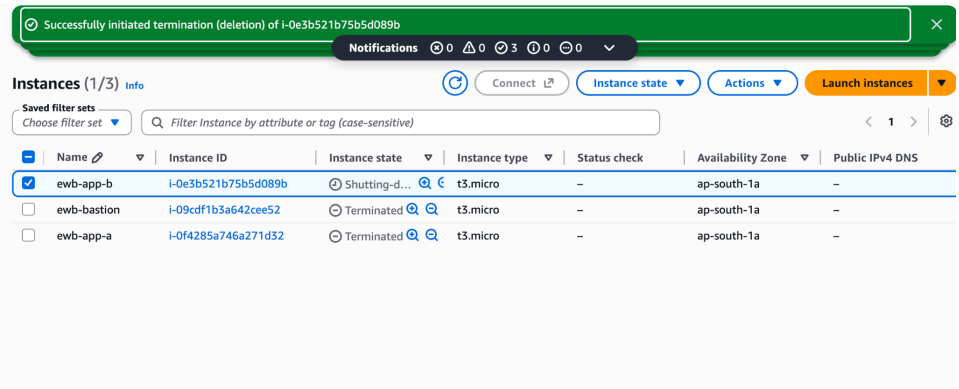
No, VPC peering does not allow VPC B to use VPC A's internet gateway. Peering supports private communication between VPCs, not shared internet access. VPC B needs its own suitable internet-access configuration.

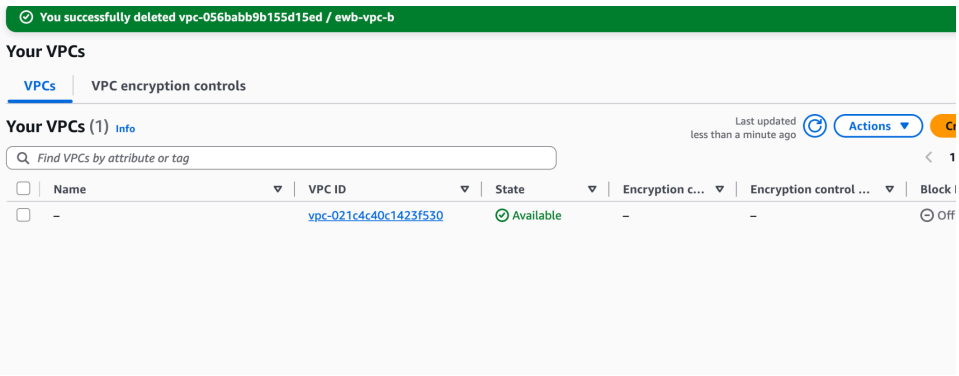
CLEANUP AND RESOURCE DELETION

The cleanup process includes:

1. Terminate **ewb-bastion**, **ewb-app-a**, and **ewb-app-b**.
2. Confirm that the lab EBS volumes have been deleted and remove any leftover lab volumes or snapshots if no longer required.
3. Release any Elastic IP address created for the project.
4. Remove the unused EC2 key pair from AWS if it is no longer needed.
5. Delete the VPC peering connection and remove the associated peering routes.
6. Delete the project security groups after their associated network interfaces have disappeared.
7. Delete the project subnets and custom route tables.
8. Detach and delete **ewb-igw-a**.
9. Delete **ewb-vpc-a** and **ewb-vpc-b**.
10. Check the AWS Billing console for remaining chargeable resources. Usage and billing information may not update immediately.







Key Learnings

- Learned to create and configure AWS VPCs, subnets, and route tables.
- Understood how VPC Peering enables private communication between two VPCs.
- Learned to configure route tables and security groups for secure connectivity.
- Understood the role of a bastion host in securely accessing private EC2 instances through SSH.
- Practiced testing connectivity using ping and troubleshooting network issues.
- Learned how to identify security-group problems by testing connectivity failure and recovery.
- Understood the importance of keeping resources secure and deleting unused AWS resources to avoid unnecessary costs.